

ValiAutoFlow — Documentación Técnica: Arquitectura del Sistema

Versión del documento: 1.0 · Última actualización: 2026-08-03

Aplica a: producción en `valiautoflow.com` (servicio `amadsite_s704ag` , `C:\Hosting\s704ag`)

Este documento describe la **arquitectura real** del sistema en producción. Es la referencia técnica para desarrollo, mantenimiento y operación. Los documentos hermanos son [DESPLIEGUE.md](#) (cómo publicar cambios) y [OPERACION.md](#) (cómo operar el día a día).

1. Resumen ejecutivo

ValiAutoFlow es un **CRM de ventas con Inteligencia Artificial multi-tenant** para agencias/concesionarios de autos. Un asesor de IA atiende WhatsApp (y otros canales) 24/7: califica leads, da seguimiento, agenda citas, cotiza y opera el negocio, mientras el usuario supervisa todo desde un panel web. Cada empresa (tenant) usa la misma instancia con **datos completamente aislados**.

Capacidades núcleo:

- Asesor IA autónomo sobre WhatsApp (Baileys) y WhatsApp Business API oficial (Meta Cloud API), Instagram DM, Facebook Messenger y Telegram.
- Pipeline de ventas, inventario de autos, contactos/CRM, calendario y citas.
- Marketing IA multicanal (Instagram, Facebook, TikTok, Telegram): generación de contenido, estrategia, campañas, publicación programada y automatizaciones por etapa/inactividad.
- Copiloto IA que opera el sistema por lenguaje natural; Agent Factory (personalidades de agentes); gBrain (memoria).
- Billing con Stripe (suscripción mensual, portal de cliente, webhooks).
- Panel de administración multi-tenant (torre de control) y ValiGuard (auditoría/seguridad).

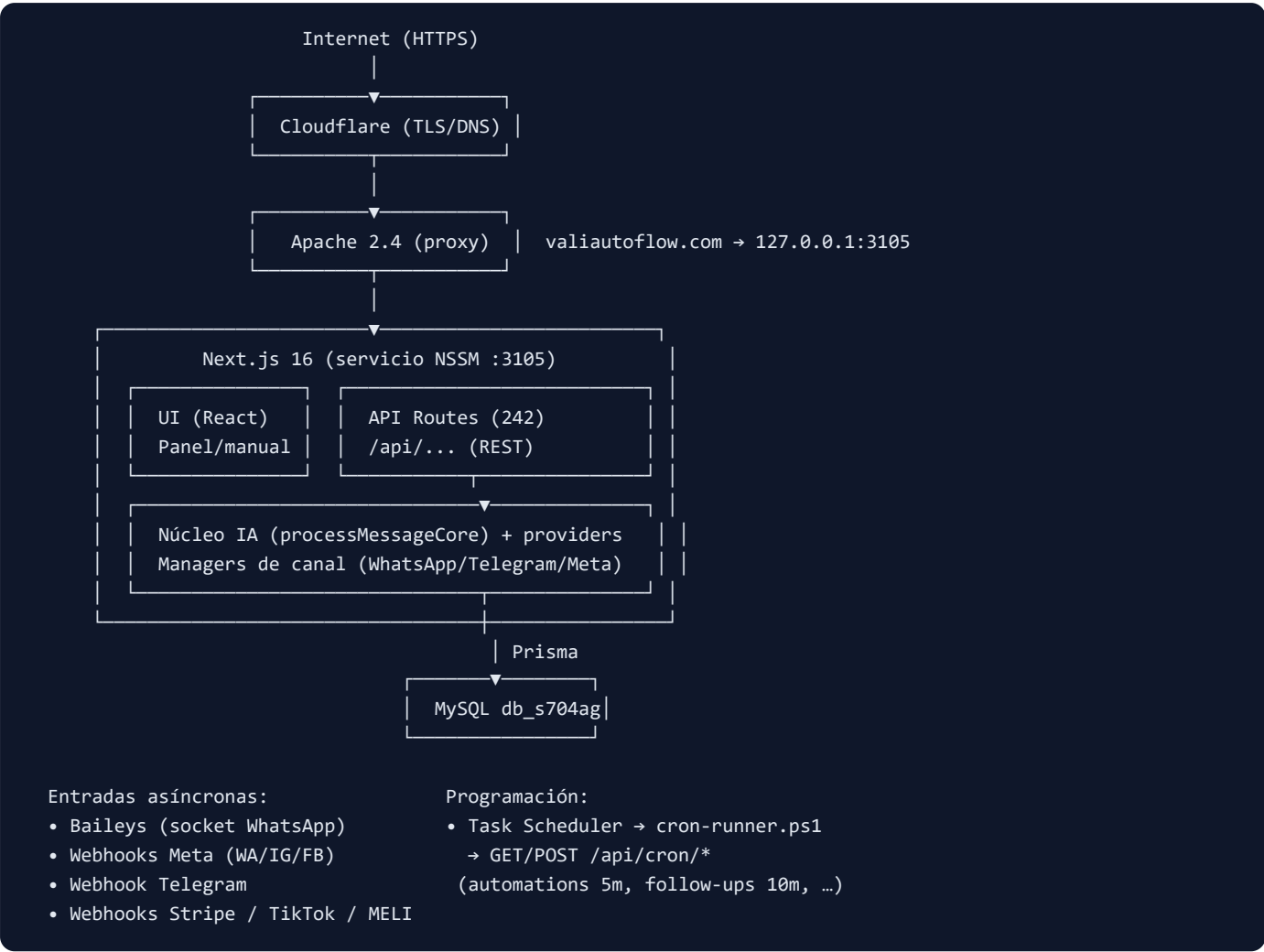
2. Stack tecnológico (real en producción)

Capa	Tecnología	Notas
Framework	Next.js 16.1 (App Router) + React 19	Renderizado híbrido (estático + server-render bajo demanda); API Routes.
Lenguaje	TypeScript	
UI	Tailwind CSS + shadcn/ui + lucide-react + sonner (toasts)	
ORM / BD	Prisma 6.19 sobre MySQL	Base <code>db_s704ag</code> . 73 modelos.
Runtime	Node.js 24	
Proceso	NSSM (servicio de Windows <code>amadsite_s704ag</code>)	Mantiene <code>next start -p 3105</code> vivo 24/7 bajo usuario restringido.
Proxy inverso	Apache 2.4	Termina TLS y hace proxy a <code>127.0.0.1:3105</code> . Timeout ~30s.
TLS / DNS	Cloudflare	Certificado y renovación gestionados por Cloudflare; DDNS por tarea programada.
IA (LLM)	MiniMax (M3 / Text-01)	Vía <code>src/lib/ai/providers.ts</code> . Proveedores <code>glm</code> / <code>zai</code> se remapean a MiniMax. Visión con MiniMax; audio (ASR) con Groq.

Capa	Tecnología	Notas
WhatsApp	Baileys 7 (no oficial, por QR) + Meta Cloud API (oficial)	Selección de canal por tenant.
Programación (cron)	Windows Task Scheduler → <code>scripts/cron-runner.ps1</code> → endpoints <code>/api/cron/*</code>	6 tareas activas.
Pagos	Stripe	Checkout, portal, webhooks.
Correo	Resend	

Nota: el contrato menciona PostgreSQL/Docker/Nginx/Let's Encrypt; la **implementación real** en producción usa MySQL/NSSM/Apache/Cloudflare. Este documento describe lo que está desplegado.

3. Vista de alto nivel



4. Estructura del proyecto

```
C:\Hosting\s704ag\  
├─ src/  
└─ app/
```

```

├─ (dashboard)/          # Panel principal (vistas por ?v=<view>)
├─ admin/                # Torre de control multi-tenant (superadmin)
├─ login, signup, ...    # Auth y páginas públicas
├─ api/                  # 242 rutas API (REST)
│  └─ cron/              # Endpoints disparados por Task Scheduler
│  └─ marketing/         # Generación, publicación, campañas, config
│  └─ whatsapp/meta/     # Cloud API: config, webhook, templates
│  └─ meta/              # Mensajería IG/FB (metaChannel)
│  └─ billing/, stripe/  # Stripe
│  └─ ...
├─ components/dashboard/ # Vistas del panel (inbox, inventory, marketing, manual...)
├─ lib/
│  └─ ai/                # providers.ts, message-processor.ts, media-understanding.ts...
│  └─ whatsapp/          # connection.ts (Baileys), meta-api.ts, channel-router.ts
│  └─ marketing/         # publish, lead-channel, stage-flows, automation-scanner...
│  └─ rbac.ts, api-auth.ts # Roles y guardas de tenant
│  └─ db.ts              # Cliente Prisma
├─ middleware.ts         # Auth de borde + rutas públicas + headers de seguridad
├─ prisma/schema.prisma  # 73 modelos
├─ public/               # Estáticos (incl. public/manual/*.png y *.mp4 del Manual)
├─ scripts/cron-runner.ps1 # Puente Task Scheduler → API
├─ docs/                 # Esta documentación
├─ .env                  # Configuración (ver §10)
└─ .next/                # Build de producción

```

5. Modelo multi-tenant (aislamiento de datos)

- El **tenant** es el modelo **Workspace** (`prisma/schema.prisma`). Cada empresa = un Workspace con su `slug` único, `ownerId` , plan y límites.
- La membresía usuario↔workspace es **WorkspaceMember** (`@@unique([userId, workspaceId])` , campo `role`). Un usuario puede pertenecer a varios workspaces.
- Todos los modelos de dominio llevan `workspaceId`** (Contact, Conversation, Deal, CatalogItem, etc.). `Message` se aísla indirectamente vía `conversationId → Conversation.workspaceId` .
- Guarda de aislamiento:** `requireWorkspace(workspaceId, userId)` en `src/lib/api-auth.ts` valida la membresía (`db.workspaceMember.findUnique`) y lanza **403** si el usuario no pertenece. Toda ruta que recibe `workspaceId` del cliente lo valida con esta función **antes** de consultar datos.
- RBAC** (`src/lib/rbac.ts`): roles `owner / admin / member (vendedor) / viewer` . `requirePermission(role, cap)` protege escrituras. Row-scoping: un `member` solo ve **sus** conversaciones/contactos asignados (`assignedTo = userId`); `owner/admin/viewer` ven todo el workspace.
- Tests de aislamiento: `src/lib/tenant-isolation.test.ts` .

Nota de diseño: el aislamiento es por **convención de ruta** (cada ruta llama `requireWorkspace`), no un filtro global de Prisma. Se cumple en las rutas y está cubierto por tests; un filtro global sería el refuerzo "cinturón y tirantes".

6. Pipeline de IA (procesamiento de mensajes)

El corazón es `src/lib/ai/message-processor.ts` → `processMessageCore(...)` . Todos los canales de entrada convergen en él (una sola "mente" para WhatsApp, Meta, Telegram, webchat):

- Entrada** (mensaje del cliente) llega por Baileys / webhook Meta / webhook Telegram.
- Comprensión multimodal** (`media-understanding.ts`): imágenes → visión (MiniMax), audio → transcripción (Groq), documentos → extracción de texto.

3. **Contexto:** se cargan workspace, contacto, conversación, perfil del lead (temperatura/arquetipo/objeción), inventario relevante, prompt del asesor (persona) y memoria (gBrain).
4. **Generación** con el proveedor de IA (`providers.ts` , MiniMax) — reasoning separado del `content` limpio.
5. **Acciones** `[CRM:...]` : el modelo emite etiquetas que el servidor ejecuta de forma determinista: `[CRM:stage:X]` (mover etapa), `[CRM:appointment:...]` (agendar), `[CRM:foto:auto]` (enviar fotos del inventario), `[CRM:cotiza]` , `[CRM:pago]` , `[CRM:factura]` , `[CRM:score]` , `[CRM:followup:Xh]` , etc.
6. **Salida:** se sanea (anti-fuga de razonamiento), se humaniza, se trocea en burbujas y se envía por el canal activo; se persiste en `Message` y se actualiza la conversación/bitácora.

Proveedor de IA (`providers.ts`): la plataforma corre en **MiniMax** (override `.env`). Por compatibilidad, cualquier llamada con proveedor `glm` / `zai` se **remapea a MiniMax**. Visión = MiniMax; ASR de audio = Groq (`GROQ_API_KEY`).

7. Canales de comunicación

7.1 WhatsApp — dos transportes, selección por tenant

- **Baileys** (`src/lib/whatsapp/connection.ts`): conexión no oficial por QR. Cada workspace tiene un `WhatsAppManager` persistente con watchdog de reconexión 24/7, dedup anti-"Bad MAC", y envío espaciado anti-baneo.
- **Meta Cloud API** (`src/lib/whatsapp/meta-api.ts`): API oficial. Config por tenant en `MetaApiConfig` (phoneNumberId, accessToken, webhookSecret, businessId). Webhook en `/api/whatsapp/meta/webhook` con verificación **HMAC** (`x-hub-signature-256` , comparación constante). Recepción multimodal (descarga y entiende imágenes/audio/doc).
- **Enrutador** (`src/lib/whatsapp/channel-router.ts`): `getActiveChannel(workspaceId)` lee `Workspace.waChannel` ; `routedSendText/Image/Template` despachan a Baileys o Meta según el canal activo.

7.2 Instagram DM + Facebook Messenger

- Modelo `MetaChannel` + `/api/meta/webhook` (verify token + routing por pageld). Conexión por OAuth de 1 clic (`/api/meta/connect` → `/api/meta/callback` , scopes de mensajería) o config manual.

7.3 Telegram

- Bot API por workspace (token en settings). Webhook `/api/telegram/webhook/[workspaceId]` . Envío a leads y publicación a canales/grupos.

8. Módulo de Marketing IA (multicanal)

- **Canales de publicación:** Facebook e Instagram (Meta Graph API), TikTok (Content Posting API), Telegram (Bot API). WhatsApp = mensajería directa a leads.
- **Generación con IA** (`/api/marketing/generate`): copy, ads, guiones de video, estrategia (con datos reales del CRM), calendario, análisis de audiencia.
- **Estudios:** Diseños (imágenes con plantillas Satori) y Video (reels con ffmpeg + TTS).
- **Publicación programada:** `ScheduledPost` + `processDueScheduled` (`schedule-runner.ts`) disparado por el cron de automatizaciones → `publishCar` (todas las redes).
- **Automatizaciones:**
- **Por etapa del pipeline** (`stage-flows.ts`): al cambiar de etapa se envía el mensaje configurado por el canal preferido del lead.
- **De leads** (`automation-scanner.ts`): triggers `days_inactive` / `temperature` / `tag` → mensaje al canal preferido (con guardas y anti-repetición).
- **Reactivación de fríos y campañas masivas** (broadcast anti-baneo).
- **Plantillas de WhatsApp (Meta):** CRUD + envío a revisión + asignación por acción (`/api/whatsapp/meta/templates` , `template-assignments`). Fuera de la ventana de 24h en Meta, `sendToLead` usa la plantilla aprobada de la acción.

9. Automatizaciones y tareas programadas (cron)

Los crons **no** corren dentro de Next; los dispara **Windows Task Scheduler** ejecutando `scripts/cron-runner.ps1` , que hace `GET http://localhost:3105/api/cron/<endpoint>` con `Authorization: Bearer ${CRON_SECRET}` .

Tarea	Frecuencia	Endpoint	Qué hace
ValiAutoFlow-Cron-Automations	5 min	<code>/api/cron/automations</code>	Automatizaciones legacy, decaimiento de score, cola de jobs, aprobaciones expiradas, disparo del piloto de marketing + publicaciones programadas, automatizaciones de leads (Pantalla 4) y publicación del calendario.
ValiAutoFlow-Cron-FollowUps	10 min	<code>/api/cron/follow-ups</code>	Procesa la cola de follow-ups (multicanal), reactivación DIB/inactividad, alertas de citas.
ValiAutoFlow-Cron-Education	15 min	<code>/api/cron/education</code>	Ejecuciones educativas programadas.
ValiAutoFlow-Cron-Briefing	30 min	<code>/api/cron/briefing</code>	Briefings.
ValiAutoFlow-Cron-WeeklyReport	2 h	<code>/api/cron/weekly-report</code>	Reporte semanal (lunes) por Telegram.
ValiAutoFlow-Cron-Snapshot	diario	<code>/api/cron/snapshot</code>	Snapshot de métricas.

Logs de cada cron: `C:\Hosting\s704ag\.cron-logs\<endpoint>.log` .

10. Autenticación y seguridad

- **Sesión:** JWT firmado (`src/lib/auth-edge.ts` , HMAC con `NEXTAUTH_SECRET`). Cookie `__Host-valiflow-session` en producción (Secure, Path=/, sin Domain).
- **Middleware** (`src/middleware.ts`): valida el JWT en el borde, define rutas públicas (webhooks, callbacks OAuth), aplica rate-limit y headers de seguridad, y protege `/admin` para superadmin. Para API sin sesión devuelve **401 JSON** (no redirige a /login).
- **Guarda de tenant + RBAC:** ver \$5.
- **Webhooks:** Meta se verifica con HMAC; los verify tokens se validan en cada ruta.
- **Secretos:** en `.env` (nunca en el cliente salvo variables `NEXT_PUBLIC_*`).

11. Integraciones externas

Integración	Uso	Config
Meta (Facebook/Instagram/WhatsApp)	Publicar, mensajería, WhatsApp Cloud API	<code>NEXT_PUBLIC_META_APP_ID</code> , <code>META_APP_SECRET</code> , <code>META_WEBHOOK_VERIFY_TOKEN</code> ; OAuth callbacks.
TikTok	Publicar videos (Content Posting API)	<code>TIKTOK_CLIENT_KEY/SECRET</code> (app de plataforma) + OAuth por tenant.
Stripe	Suscripción, portal, webhooks	<code>STRIPE_SECRET_KEY</code> , <code>STRIPE_PUBLIC_KEY</code> , <code>STRIPE_WEBHOOK_SECRET</code> .

Integración	Uso	Config
Google	Calendar por usuario (OAuth)	<code>GOOGLE_CLIENT_ID/SECRET</code> .
Mercado Libre	Publicar/gestionar autos	<code>MELI_APP_ID/SECRET</code> + OAuth por tenant.
MiniMax	LLM + visión + TTS	<code>MINIMAX_API_KEY</code> , <code>MINIMAX_*_MODEL</code> , <code>MINIMAX_GROUP_ID</code> .
Groq	Transcripción de audio (ASR)	<code>GROQ_API_KEY</code> .
Resend	Correo transaccional	<code>RESEND_API_KEY</code> , <code>EMAIL_FROM</code> .

Variables de entorno principales (`.env`)

`DATABASE_URL` , `NEXTAUTH_SECRET` , `NEXTAUTH_URL` , `NEXT_PUBLIC_APP_URL` , `PORT` (3105), `CRON_SECRET` , `WORKER_KEY` , `HEALTH_INTERNAL_KEY` , `WHATSAPP_AUTH_DIR` , `AI_PROVIDER_OVERRIDE` , más las llaves de cada integración de arriba. **Nunca** commitear `.env` .

12. Modelo de datos (resumen)

73 modelos Prisma. Los principales por área:

- **Tenancy/Auth:** `Workspace` , `WorkspaceMember` , `User` , `Session` , `Invitation` , `Subscription` .
- **CRM:** `Contact` , `Conversation` , `Message` , `LeadProfile` , `ContactTimelineEvent` , `Deal` , `Pipeline` , `PipelineStage` .
- **Inventario:** `CatalogItem` (autos; estado y fotos en `metadata` JSON).
- **IA/Agentes:** `Agent` , `AgentPersona` , `AgentMemory` , `AiTrainingExample` , gBrain.
- **Automatización:** `Automation` , `MarketingAutomation` , `FollowUpTask` , `FollowUpRule` .
- **Marketing:** `MarketingCampaign` , `MarketingContent` , `ContentCalendarEntry` , `ScheduledPost` , `MarketingPost` , `MarketingBotConfig` .
- **Canales:** `MetaApiConfig` (WhatsApp Cloud), `MetaChannel` (IG/FB), `MetaMessageTemplate` .
- **Citas:** `Appointment` (+ integración Google Calendar).

13. Referencias cruzadas

- **Publicar cambios:** [DESPLIEGUE.md](#)
- **Operar el día a día:** [OPERACION.md](#)
- **Onboarding de agencias:** [ONBOARDING-AGENCIA.md](#)